

Building a Serverless Data Lake with AWS S3, Athena, and Glue

Step 1: S3 Bucket Creation

A new Amazon S3 bucket named data-bucket-1493854 was created to serve as the foundation for the data lake. The bucket was provisioned in the US East (N. Virginia) region (us-east-1) on December 22, 2025, at 4:52:50 PM (UTC-06:00). This bucket will be used to store both raw data and query results throughout the project.

	data-bucket-1493854	US East (N. Virginia) us-east-1	December 22, 2025, 16:52:50 (UTC-06:00)
---	-------------------------------------	---------------------------------	---

Step 2: Folder Structure Setup

Within the newly created S3 bucket, two distinct folders were established:

- cache/ – This folder is dedicated to storing the results of Athena queries.
- ccdata/ – This folder holds the CSV data file that contains the main dataset for analysis.

☐	 cache/	Folder	-	-	-
☐	 ccdata/	Folder	-	-	-

Step 3: Data Upload

The primary data file, ccspend.csv, was uploaded to the ccdata/ folder within the S3 bucket. The file size is 148.8 KB, and it is stored at s3://data-bucket-1493854/ccdata/. The file type is application/vnd.ms-excel, indicating it is in CSV format.

☐	ccspend.csv	-	application/vnd.ms-excel	148.8 KB
--------------------------	-------------	---	--------------------------	----------

Step 4: Athena Cache Configuration

Athena was configured to use the cache/ folder in the S3 bucket as the location for storing query execution results. This setup allows Athena to write output and intermediate results to a designated area within S3, facilitating efficient, serverless querying.

Step 5: Database Creation

An SQL command was executed to create a new database named appdata in the AWS Glue Data Catalog. This logical database serves as an organizational layer for tables and metadata, enabling efficient management and discovery of datasets within AWS services.

```
create database appdata;
```

Step 6: External Table Definition

An external table named ccspend was defined in the appdata database using the following SQL statement:

```
CREATE EXTERNAL TABLE IF NOT EXISTS appdata.ccspend (  
    year_month bigint,  
    agency_number bigint,  
    last_name string,  
    first_name string,  
    agency_name string,  
    amount double,  
    vendor string,  
    txn_date string,  
    posted_date string,  
    description string  
)  
  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe'
```

```
WITH SERDEPROPERTIES ('serialization.format' = ',', 'field.delim' = ',')
```

```
LOCATION 's3://data-bucket-1493854/ccdata/'
```

```
TBLPROPERTIES ('has_encrypted_data'='false');
```

This step defined the table schema, including column names and data types, and specified the CSV format with a comma delimiter. The table points directly to the location of the CSV file in S3, enabling a data lake pattern where data is queried without being loaded into a database. The table is created as a reference, and no data is physically moved or copied.

Step 7: Data Query – Preview

A preview query was run to examine the data. The following SQL command was used:

```
select * from ccspend limit 10;
```

The query returned 10 rows of credit card spending data, displaying columns such as year_month (e.g., 201307 for July 2013), agency_number (e.g., 1000), last and first names (e.g., Mason, Massey, Mauro-Herrera), agency_name (e.g., GENERAL PURCHASE, ROOM CHARGES), amount (ranging from \$10.10 to \$890.00), vendor (e.g., NACAS, SHERATON HOTEL, SEARS.COM, WAL-MART), and transaction and posted dates from July 2013.

#	year_month	agency_number	last_name	first_name	agency_name	amount	vendor	txn_date
1	201307	1000	Mason	C	GENERAL PURCHASE	890.0	NACAS	07/30/2013 12:00:00 AM
2	201307	1000	Mason	C	ROOM CHARGES	368.96	SHERATON HOTEL	07/30/2013 12:00:00 AM
3	201307	1000	Massey	J	GENERAL PURCHASE	165.82	SEARS.COM 9300	07/29/2013 12:00:00 AM
4	201307	1000	Massey	T	GENERAL PURCHASE	96.39	WAL-MART #0137	07/30/2013 12:00:00 AM
5	201307	1000	Mauro-Herrera	M	HAMMERMILL COPY PLUS COPY EA	125.96	STAPLES DIRECT	07/30/2013 12:00:00 AM
6	201307	1000	Mauro-Herrera	M	GENERAL PURCHASE	394.28	KYOCERA DOCUMENT SOLUTION	07/29/2013 12:00:00 AM
7	201307	1000	Mays	B	GENERAL PURCHASE	19.87	WAL-MART #0137	07/30/2013 12:00:00 AM
8	201307	1000	Mays	B	GENERAL PURCHASE	10.1	DAYLIGHT DONUTS	07/29/2013 12:00:00 AM
9	201307	1000	McCarthy	D	GENERAL PURCHASE	142.6	UFIRST LAUNDRY SVCS	07/30/2013 12:00:00 AM
10	201307	1000	McClurg	L	AT&T SERVICE PAYMENT ITM	29.99	AT&T DATA	07/30/2013 12:00:00 AM

Step 8: Data Analysis – Aggregation Query

An analytical query was performed to aggregate credit card transactions by agency or category. The following SQL was executed:

```
select count(1) NumTxn, agency_name
```

from ccspend

group by agency_name

having count(1) > 5;

The results showed the number of transactions for each category, filtered to include only those with more than five transactions:

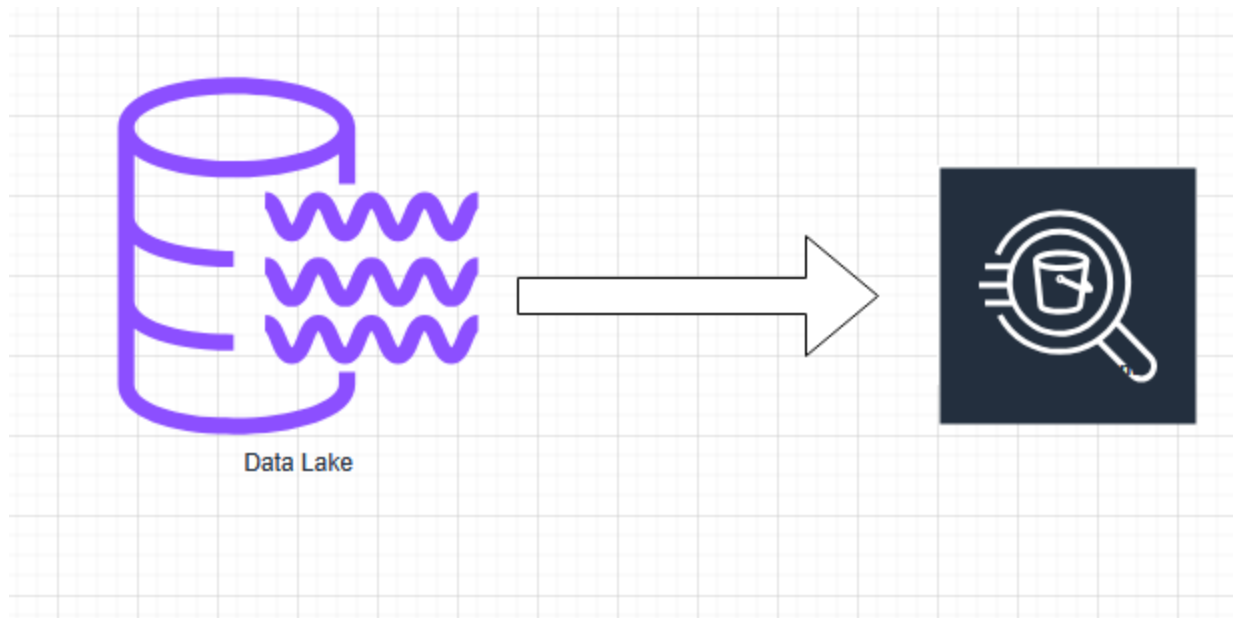
1. GENERAL PURCHASE: 679 transactions
2. ROOM CHARGES: 32 transactions
3. AIR TRAVEL: 10 transactions
4. JANITORIAL SUPPLIES NMB: 7 transactions

This query demonstrates how SQL analytics can be performed directly on data stored in S3, leveraging Athena's serverless capabilities.

```
1 select count(1) NumTxn, agency_name
2 from ccspend
3 group by agency_name
4 having count(1) > 5;
```

#	▼	NumTxn	▼	agency_name
1		679		GENERAL PURCHASE
2		32		ROOM CHARGES
3		10		AIR TRAVEL
4		7		JANITORIAL SUPPLIES NMB

Architecture:



Project Summary

This project successfully demonstrated the following key concepts:

- **Data Lake Architecture:** CSV data is stored in S3 without being loaded into a traditional database.
- **Schema-on-Read:** The table structure is defined at query time, not when data is stored.
- **Serverless Querying:** Athena is used to run SQL queries without the need to manage dedicated infrastructure.
- **AWS Glue Integration:** Metadata about the tables and databases is maintained in the Glue Data Catalog.
- **Cost-Effective Analytics:** Costs are incurred only for queries that are executed, with no charges for data storage or idle compute resources.

Key Achievement: Structured data stored in S3 was queried using standard SQL, effectively treating the data lake as a database and enabling efficient, scalable analytics.